Exercises for
Introduction to Machine Learning
Linda-Sophie Schneider, Johannes Enk, Fengyu Han, Gia Long Nguyen, Lukas Huettner, Yangbin Peng
Task 5: deadline: 27th July 2025

Pattern Recognition Lab

Exercise 05: KNN and Logistic Regression

In this task, you are supposed to implement a classification pipeline using the K-Nearest Neighbors (KNN) algorithm and Logistic Regression. You will work with the Kaktovic dataset, which contains images of 11 different symbols. The goal is to classify these symbols. Since Logistic Regression is not easy to implement for this task, you will implement a KNN classifier from scratch and use the Logistic Regression function provided by sklearn.

Exercise 1 Read first

The python files contain information which will help you solving this exercise. Please, read quickly[1] the python files in order to have an idea of their content before proceeding further.

Exercise 2 KNN Classifier Implementation

In this task, you will implement a K-Nearest Neighbors (KNN) classifier from scratch in `knn.py`. Your classifier should support both Euclidean and cosine distance metrics for measuring similarity between samples. The class should be able to fit on training data, predict labels for new samples, and optionally visualize the nearest neighbors for a given test image.

- **fit(X, y):** Store the training data and corresponding labels for later use during prediction.
- **predict(X):** For each test sample, compute its distance (or similarity) to all training samples using the selected metric. Identify the k nearest neighbors, perform a majority vote to determine the predicted label, and return the predictions. If visualization is enabled, display the test image alongside its nearest neighbors.
- **Distance Metrics:** Implement both Euclidean distance and cosine similarity (converted to distance) as options for neighbor search.
- **Visualization:** If `plot_neighbors` is set to `True`, use the provided `plot_knn_neighbors` function to visualize the test image and its nearest neighbors.

Make sure your implementation is efficient and handles edge cases, such as ties in majority voting or division by zero in cosine similarity. Test your classifier on the provided dataset and compare its performance using different metrics and values of k.

[1] Of course, read these files more carefully while solving the exercises!

Exercise 3 Confusion Matrix Implementation

Implement a function to compute the confusion matrix for a multi-class classification problem using only numpy. Your function should:

- Accept two input arrays: `y_true` (ground truth labels) and `y_pred` (predicted labels), both of shape $(n_samples,)$.
- Infer the number of classes from the unique values present in `y_true` and `y_pred`, assuming class labels are consecutive integers starting from 0.
- Initialize a square confusion matrix of shape (num_classes, num_classes) with zeros.
- Populate the confusion matrix such that the entry at position (i, j) contains the count of samples with true label i and predicted label j.
- Return the resulting confusion matrix as a numpy ndarray.

Exercise 4 Submission Instructions and Plot Saving

To successfully complete this exercise, you must ensure that all provided tests run without errors. In addition to running the code, you are required to save and upload all within the test generated plots for grading, as there will be no time for in-person presentations.

- **KNN Comparison Plots:** Save and upload 11 different plots generated during your KNN experiments.
- **Confusion Matrix Plots:** Save and upload 2 plots showing confusion matrices for your classification results.
- **Code Submission:** Upload all your code files along with the plots.

We highly encourage you to complete this exercise, as it is excellent preparation for the exam. Remember, your submission should include all required plots and code files for full credit.